Exam 2
CpE 3150-Intro to Microcontrollers and Embedded Systems

This exam is an individual activity. The exam is open textbook and other class materials posted on Canvas for this course. Other outside materials are NOT permitted. Answers given that are identified as coming from outside sources or as shared work with another classmate will treated as academic dishonesty. All problems with answers identified as dishonest will be scored as 0 points, and there may be a referral to the ECE Department Chair for academic dishonesty.

Show all work. Partial credit will be given based on work shown.

I completed this exam on my own following the guidelines above and did not obtain or provide assistance to anyone else taking this exam. This includes not seeking online assistance from others.

_____ _____

Signature Date

(50 points) Part 1 of the Exam is a quiz (Exam 2: Part 1) on Canvas. This part of the exam consists of True/False and Multiple Choice problems. No work is required to be shown for this part of the exam. You may submit any work for this part of the exam on separate sheet(s) with the problem(s) labeled.

(**50 points**) Part 2 of the Exam is given as follows. Showing your work is necessary for partial credit. You may print out this part of the exam and write your solutions, scan them, and submit the scanned document on Canvas for Exam 1. You may also provide your solutions (with work shown) in this Word document and submit on Canvas.

Both parts of the exam are subject to the integrity statement that you are signing above. Both parts of the exam must be completed (part 1 on Canvas) and submitted on Canvas (part 2 here) by the exam deadline. Best wishes with the exam!

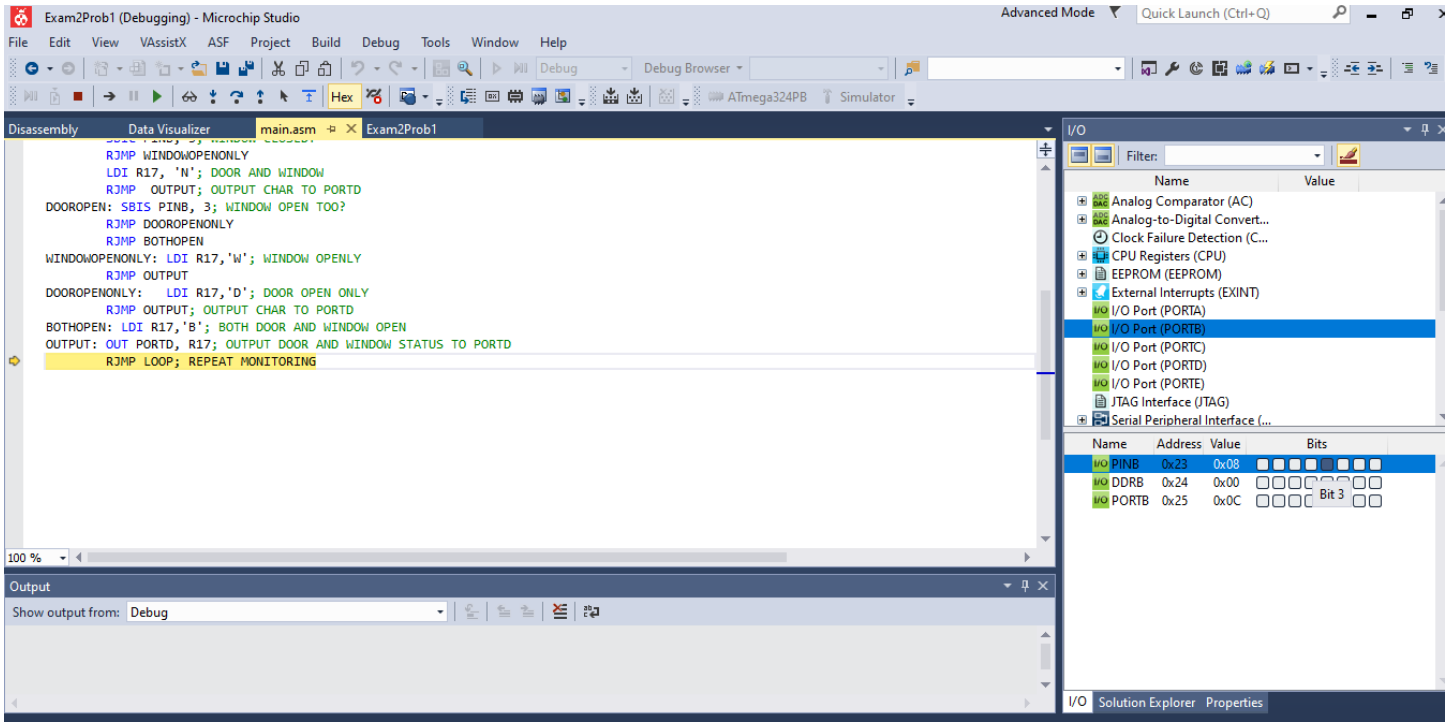
1. Answer the questions for parts a and b below based on the ATmega32 (textbook) microcontroller.

a. **(10 Points)** Assume that PORTB, BIT 2 (PB2) and BIT 3 (PB3) are inputs connected to a door sensor (PB2) and a window sensor (PB3) and that PORTD is an output connected to a display terminal that shows a single ASCII character that displays the current status of the door and window sensors in a vehicle. Write an assembly language program to use a conditional loop to continuously read the status of the door and window sensors (0 = closed, 1 = open). If the door is open, output an ASCII 'D' to PORTD. If the window is open, output an ASCII 'W' to PORTD. If both the door and windows are open, then output an ASCII 'B' to PORTD. Otherwise (neither the door nor the window is open), output an ASCII 'N' to PORTD.

```
.ORG 0
 CBI DDRB, 2
 SBI PORTB, 2; DOOR SENSOR INPUT + PULLUP
 CBI DDRB, 3
 SBI PORTB, 3; WINDOW SENSOR INPUT + PULLUP
 LDI R16, 0XFF
 OUT DDRD, R16; PORTD OUTPUT

 LOOP: SBIC PINB, 2; DOOR CLOSED?
              RJMP DOOROPEN
              SBIC PINB, 3; WINDOW CLOSED?
              RJMP WINDOWOPENONLY
              LDI R17, 'N'; DOOR AND WINDOW
              RJMP  OUTPUT; OUTPUT CHAR TO PORTD
DOOROPEN: SBIS PINB, 3; WINDOW OPEN TOO?
              RJMP DOOROPENONLY
              RJMP BOTHOPEN
WINDOWOPENONLY:      LDI R17,'W'; WINDOW OPENLY
              RJMP OUTPUT
DOOROPENONLY: LDI R17,'D'; DOOR OPEN ONLY
              RJMP OUTPUT; OUTPUT CHAR TO PORTD
BOTHOPEN: LDI R17,'B'; BOTH DOOR AND WINDOW OPEN
OUTPUT:       OUT PORTD, R17; OUTPUT DOOR AND WINDOW STATUS TO PORTD
       RJMP LOOP; REPEAT MONITORING
```

b. **(5 Points)** Create an assembly language Microchip Studio workspace for the ATmega324pb for your program in part a. Submit a screenshot of the workspace single step simulating the workspace with the bit values of PORTB BIT 2 (door sensor) and PORTB, BIT 3 (window sensor) set (both pin values are 1s). Show the value of PORTD in the simulation window. You just need to capture a screenshot of the simulation when PB2 and PB3 are set (1s) with the values of PORTC shown. In the simulator, how do you change the value of the door and window sensor inputs?



DEBUG\WINDOWS\I/O => DOUBLE CLICK ON PORTB => SINGLE STEP THE PROGRAM => TO CHANGE DOOR AND WINDOW SENSOR VALUES CLICK ON PINB2 (DOOR) OR PINB3 (WINDOW) => THE OUTPUT STATUS IS SHOWN UPON EXECUTING THE RJMP LOOP STATEMENT AND CAN BE SEEN BY DOUBLE CLICKING ON PORTD IN I/O AND LOOK AT THE VALUE OF PORTD

2. Answer the questions for parts a and b below.

a. (12 Points) Write an assembly program using Timer 0 to continuously generate a 1.2 kHz square wave on PORTC, bit 3 (PC3) using Timer 0 overflow (TOV0). Assumer fosc = 16 MHz.

Twave = 1/fwave = 1/1200 s = 833.33 us Thalf = 416.67 us = tdelay = n tMC
tMC = (1 cc/1MC)(1s/16(10^6) cc)
n = tdelay/tMC = 6666.72 (> 255 so need prescaler)
for clk/64 => 6666/64 = 104.2 (choose n = 104)

```
.ORG 0
SBI DDRC, 3;  PC3 OUTPUT
LDI R16, -104; PRELOAD FOR TCNT0
LDI R17, 0b00000011; TCCR0 CLK/64 AND NORMAL OPERATION
LDI R18, 0b00001000; BIT 3 FOR TOGGLING PC3
LDI R21, 0
LDI R22, 1<<TOV0
MAIN:   IN R19, PORTC; GET CURRENT VALUE OF PC (BIT 3 IN PARTICULAR)
        EOR R19, R18; TOGGLE BIT 3
        OUT PORTC, R19; UPDATE PC3
        RCALL DELAY; HALF-PERIOD TIME DELAY
        RJMP MAIN
.ORG 0x50
DELAY:  OUT TCNT0, R16; LOAD TCNT0
        OUT TCCR0, R17, TURN ON TIMER0, NORMAL, CLK/64
WAIT:   IN R20, TIFR; GET STATUS OF TOV0
        SBRS R20, TOV0
        RJMP WAIT; WAIT FOR TOV0 TO BE SET FOR HALF PERIOD TIME DELAY
        OUT TCCR0, R21; TURN OFF TIMER 0
        OUT TIFR, R22; RESET TOV0
        RET
```

b. (8 Points) Copy your solution from part a into the space below. **Circle the instructions that need to be modified** to continuously generate a 1.2 kHz square wave on PORTC, bit 3 (PC3) using the CTC mode for Timer 0. Assumer fosc = 16 MHz. **Write the instruction sequence to replace the circled instructions** that uses the CTC mode to generate the 1.2 kHz square wave on PC3.

SOLUTION TO PART A-LINES TO CHANGE FOR CTC MODE

```
.ORG 0
SBI DDRC, 3;  PC3 OUTPUT
LDI R16, -104; PRELOAD FOR TCNT0
```

```
LDI R17, 0b00000011; TCCR0 CLK/64 AND NORMAL OPERATION
LDI R18, 0b00001000; BIT 3 FOR TOGGLING PC3
LDI R21, 0
LDI R22, 1<<TOV0
MAIN:   IN R19, PORTC; GET CURRENT VALUE OF PC (BIT 3 IN PARTICULAR)
        EOR R19, R18; TOGGLE BIT 3
        OUT PORTC, R19; UPDATE PC3
        RCALL DELAY; HALF-PERIOD TIME DELAY
        RJMP MAIN
.ORG 0x50
DELAY:  OUT TCNT0, R16; LOAD TCNT0
        OUT TCCR0, R17, TURN ON TIMER0, NORMAL, CLK/64
WAIT:   IN R20, TIFR; GET STATUS OF TIMER FLAG
        SBRS R20, TOV0
        RJMP WAIT; WAIT FOR TIMER FLAG TO BE SET FOR HALF PERIOD TIME DELAY
        OUT TCCR0, R21; TURN OFF TIMER 0
        OUT TIFR, R22; RESET TIMER FLAG
        RET
```

SOLUTION TO PART B

```
.ORG 0
SBI DDRC, 3;  PC3 OUTPUT
LDI R16, 103; PRELOAD FOR OCR0
LDI R17, 0b00001011; TCCR0 CTC, CLK/64 OPERATION
LDI R18, 0b00001000; BIT 3 FOR TOGGLING PC3
LDI R21, 0
LDI R22, 1<<OCF0
OUT OCR0, R16; OCR0 = 103
OUT TCNT0, R21; TCNT0 = 0
MAIN:   IN R19, PORTC; GET CURRENT VALUE OF PC (BIT 3 IN PARTICULAR)
        EOR R19, R18; TOGGLE BIT 3
        OUT PORTC, R19; UPDATE PC3
        RCALL DELAY; HALF-PERIOD TIME DELAY
        RJMP MAIN
.ORG 0x50
DELAY:  OUT TCCR0, R17, TURN ON TIMER0, CTC, CLK/64
WAIT:   IN R20, TIFR; GET STATUS OF TIMER FLAG
        SBRS R20, OCF0
        RJMP WAIT; WAIT FOR TIMER FLAG TO BE SET FOR HALF PERIOD TIME DELAY
        OUT TCCR0, R21; TURN OFF TIMER 0
        OUT TIFR, R22; RESET TIMER FLAG
        RET
```

3. Given the look up table below. Answer the questions for parts a and b below.

```
; look up table for assembly language program
.ORG 0x150
NumString: .DB “012345”
```

a. (5 Points) List the code memory locations associated with each byte of the look up table above.

0x2A0 ‘0’
0x2A1 ‘1’
0x2A2 ‘2’
0x2A3 ‘3’
0x2A4 ‘4’
0x2A5 ‘5’

b. (10 Points) Write an assembly language program to read each value of the lookup table below and output it to PORTA. Use indirect addressing (pointer register Z) to access/read each value of the lookup table.

```
.ORG 0
LDI R16, HIGH(NumString<<1)
OUT ZH, R16;
LDI R16, LOW(NumString<<1)
OUT ZL, R16; INITIALIZE Z TO BYTEWISE ADDRESS TO FIRST BYTE OF NumString
LDI R17, 0xFF
OUT DDRA, R17; PA OUTPUT
LDI R17, 6; COUNTER FOR NUMBER OF BYTES IN NumString
LOOP:       LPM R18, Z+;  GET CURRENT BYTE POINTED TO IN NumString by Z, THEN
                          ; INCREMENT Z
            OUT PORTA, R18; OUTPUT CURRENT BYTE TO PA
            DEC R17; DECREMENT CHARACTER COUNT
            BRNE LOOP; SEE IF DONE OUTPUTTING ALL BYTES OF NumString TO PA
HERE:  RJMP HERE; DONE OUTPUTTING ALL BYTES OF NumString TO PA, NOW DO NOTHING
```

```
.ORG 0x150
NumString: .DB “012345”
```